

How to Build Multi-Language React Apps with Internationalization (i18n)

Have you ever built a React app only to realize that users from different countries can't use it effectively because it only supports English?

In this tutorial, you'll learn how to build truly global React application using internationalization (i18n). You'll discover how to manage translations, handle date and number formatting for different locales, and implement language switching – all while maintaining a clean, scalable codebase.

What is Internationalization (i18n)?

Internationalization (i18n) is the process of designing and developing your application so it can be easily adapted to different languages and regions without requiring engineering changes.

The term "i18n" comes from the fact that there are 18 letters between the first "i" and the last "n" in "internationalization".

Key aspects of i18n include:

- **Translation:** Converting text strings to different languages
- **Localization:** Adapting content for specific regions (dates, numbers, currencies)
- **Pluralization:** Handling singular/plural forms correctly in each language

Why i18n Matters for Your React App

Here are several compelling reasons why you should internationalize your React application:

- **Global Reach:** Reach users in their native language and expand to new markets
- **Better User Experience:** Users prefer apps in their own language – studies show 75% of users prefer buying products in their native language
- **Competitive Advantage:** Many apps lack proper internationalization, giving you an edge
- **Legal Requirements:** Some countries require apps to be available in local languages

- **SEO Benefits:** Multi-language content improves search rankings in different regions
- **Increased Conversions:** Users are more likely to complete actions when content is in their language

Prerequisites

To follow along with this tutorial, you should have:

- Basic knowledge of React and React Hooks
- Node.js and npm installed on your machine
- A code editor like VS Code

Project Setup

Let's create a new React application using Vite:

```
npm create vite@latest react-i18n-demo -- --template react
cd react-i18n-demo
```

Next, install the required i18n packages:

```
npm install i18next react-i18next i18next-browser-languagedetector
```

Here's what each package does:

- **i18next:** Core internationalization framework
- **react-i18next:** React bindings for i18next
- **i18next-browser-languagedetector:** Automatically detects user's preferred language

Now start the development server:

```
npm run dev
```

Your app should now be running on <http://localhost:5173/>

Note that, in this tutorial we will not be using any styling library or framework like Tailwind CSS.

For simplicity, we will be defining styles inline however, in your real application, you should never define styles inline. Always separate out into their own CSS files or use CSS framework like Tailwind CSS.

So delete the `src/App.css` file and replace the content of `src/index.css` with the following content:

```
:root {  
  font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;  
  line-height: 1.5;  
  font-weight: 400;  
}
```

The Problem: Hardcoded Strings

Before we implement i18n, let's look at a typical React component with hardcoded strings:

```

import { useState } from 'react';

function ProductCard({ product }) {
  return (
    <div style={{
      padding: '20px',
      border: '1px solid #ddd',
      borderRadius: '8px'
    }}>
      { /* ❌ Hardcoded English strings */ }
      <h3>{product.name}</h3>
      <p>Price: ${product.price}</p>
      <p>In stock: {product.stock} items</p>

      {product.stock === 0 && (
        <div style={{ color: 'red' }}>Out of stock</div>
      )}

      {product.stock === 1 && (
        <div style={{ color: 'orange' }}>Only 1 item left!</div>
      )}

      {product.stock > 1 && (
        <div style={{ color: 'green' }}>In stock</div>
      )}

      <button>Add to cart</button>
    </div>
  );
}

export default ProductCard;

```

This approach has several problems:

- **No Translation Support:** All text is in English only
- **Maintenance Nightmare:** Changing text requires editing multiple components
- **No Pluralization:** Handling "1 item" vs "2 items" is hardcoded
- **No Localization:** Date and number formats are fixed
- **Difficult Scaling:** Adding new languages requires modifying every component

Let's fix these issues with proper internationalization.

The Solution: react-i18next

The most popular and powerful solution for React internationalization is **react-i18next**.

It provides:

- Simple API for translations
- Automatic language detection
- Pluralization support
- Nested translations
- Formatting for dates, numbers, and currencies
- Lazy loading of translation files
- TypeScript support

How to Create Translation Files

For larger applications, it's better to organize translations in separate JavaScript files.

So, create a new `utils` folder inside `src` folder and inside it create `locales` folder where translations will be stored like this:

```
utils/  
  locales/  
    en/  
      translation.js  
    es/  
      translation.js  
    fr/  
      translation.js
```

Here's an example `utils/locales/en/translation.js` file for english translations with the following content:

```
export default {
  en: {
    translation: {
      "product.price": "Price",
      "product.inStock": "In stock: {{count}} items",
      "product.outOfStock": "Out of stock",
      "product.onlyOneLeft": "Only 1 item left!",
      "product.addToCart": "Add to cart",

      "profile.welcome": "Welcome, {{name}}!",
      "profile.memberSince": "Member since: {{date}}",
      "profile.loyaltyPoints": "You have {{points}} loyalty points",
      "profile.orders_zero": "You have no orders",
      "profile.orders_one": "You have 1 order",
      "profile.orders_other": "You have {{count}} orders",
      "profile.editProfile": "Edit profile",
    },
  },
};
```

Similarly, create `utils/locales/es/translation.js` file for spanish translations with the following content:

```
export default {
  es: {
    translation: {
      "product.price": "Precio",
      "product.inStock": "En stock: {{count}} artículos",
      "product.outOfStock": "Agotado",
      "product.onlyOneLeft": "¡Solo queda 1 artículo!",
      "product.addToCart": "Añadir al carrito",

      "profile.welcome": "¡Bienvenido, {{name}}!",
      "profile.memberSince": "Miembro desde: {{date}}",
      "profile.loyaltyPoints": "Tienes {{points}} puntos de fidelidad",
      "profile.orders_zero": "No tienes pedidos",
      "profile.orders_one": "Tienes 1 pedido",
      "profile.orders_other": "Tienes {{count}} pedidos",
      "profile.editProfile": "Editar perfil",
    },
  },
};
```

Similarly, create `utils/locales/fr/translation.js` file for french translations with the following content:

```
export default {
  fr: {
    translation: {
      "product.price": "Prix",
      "product.inStock": "En stock: {{count}} articles",
      "product.outOfStock": "Rupture de stock",
      "product.onlyOneLeft": "Plus qu'un article !",
      "product.addToCart": "Ajouter au panier",

      "profile.welcome": "Bienvenue, {{name}} !",
      "profile.memberSince": "Membre depuis : {{date}}",
      "profile.loyaltyPoints": "Vous avez {{points}} points de fidélité",
      "profile.orders_zero": "Vous n'avez aucune commande",
      "profile.orders_one": "Vous avez 1 commande",
      "profile.orders_other": "Vous avez {{count}} commandes",
      "profile.editProfile": "Modifier le profil",
    },
  },
};
```

How to Set Up react-i18next

Now, create a `i18n.js` file inside `utils` folder and add the following contents inside it:

```

import i18n from "i18next";
import LanguageDetector from "i18next-browser-languagedetector";
import { initReactI18next } from "react-i18next";
import enTranslations from "../locales/en/translation";
import esTranslations from "../locales/es/translation";
import frTranslations from "../locales/fr/translation";

const resources = {
  // English translations
  ...enTranslations,
  // Spanish translations
  ...esTranslations,
  // French translations
  ...frTranslations,
};

// Initialize i18next
i18n
  .use(LanguageDetector) // Detect user language
  .use(initReactI18next) // Pass to react-i18next
  .init({
    resources,
    fallbackLng: "en", // Default language if detection fails

    interpolation: {
      escapeValue: false,

      // Custom formatter for dates and currencies
      format: (value, format, lng) => {
        if (format === "currency") {
          return new Intl.NumberFormat(lng, {
            style: "currency",
            currency: lng === "en" ? "USD" : lng === "es" ? "EUR" : "EUR",
          }).format(value);
        }

        if (format === "date") {
          return new Intl.DateTimeFormat(lng, {
            year: "numeric",
            month: "long",
            day: "numeric",
          }).format(value);
        }

        return value;
      },
    },
  });

```



```
export default i18n;
```

Here, we have imported all the translations files and added inside the `resources` object and passed it to `init` function of `i18n`.

Also, note that we're using `i18next-browser-languagedetector` package which gives us `LanguageDetector` which we're passing to `use` function so it automatically detects language of the user who's browsing the application.

It's important to know that we're passing an object with `resources` property to `init` function. It has to be called `resources` as it's predefined property which `init` function expects.

Now import this configuration in your `src/main.jsx` file:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";
import "./utils/i18n";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Understanding Translation Variables and Interpolation

Before diving into using translations in components, it's crucial to understand how variables and interpolation work in `i18next`. This will make it much easier to create dynamic, reusable translations.

What is Interpolation?

Interpolation is the process of inserting dynamic values into translation strings. Instead of having separate translations for every possible variation, you can use placeholders that get replaced with actual values at runtime.

For example, instead of creating separate translations like:

- "Welcome, John!"
- "Welcome, Sarah!"
- "Welcome, Michael!"

You create one translation with a placeholder:

- "Welcome, {{name}}!"

Basic Variable Interpolation

The most basic form of interpolation uses double curly braces `{{variableName}}` as placeholders:

```
// import useTranslation hook
import { useTranslation } from 'react-i18next';

// use it inside component
const { t } = useTranslation();

// Translation string
"Welcome": "Welcome, {{name}}!"

// Usage in code
t('Welcome', { name: 'John' })
// Output: "Welcome, John!"

// Translation string
"product.inStock": "In stock: {{count}} items"

// Usage in code
t('product.inStock', { count: 10 })
// Output: "In stock: 10 items"
```

When you call the `t` function, you pass an object as the second argument containing the values you want to interpolate.

How to Use Translations in Components

Now let's update our code to use translations instead of hardcoded strings.

Using the useTranslation Hook

The `useTranslation` hook is the primary way to access translations in functional components.

So, create a `components` folder inside `src` folder and create `ProductCard.jsx` file inside it and add the following contents inside it:

```
import { useTranslation } from "react-i18next";

function ProductCard({ product }) {
  const { t } = useTranslation();
  const zeroLeft = product.stock === 0;
  const oneLeft = product.stock === 1;

  return (
    <div>
      <h3>{product.name}</h3>

      <p>
        {t("product.price")}: ${product.price}
      </p>

      <p>{t("product.inStock", { count: product.stock })}</p>

      {zeroLeft && (
        <div style={{ color: "red" }}>{t("product.outOfStock")}</div>
      )}

      {oneLeft && (
        <div style={{ color: "orange" }}>{t("product.onlyOneLeft")}</div>
      )}

      <button disabled={zeroLeft}>{t("product.addToCart")}</button>
    </div>
  );
}

export default ProductCard;
```

As you can see in the above code, we're using the properties from the `translation.js` file while calling the `t` function to get specific translation text.

Now, let's use this component and display it on the UI.

Open `src/App.jsx` file and replace it with the following content:

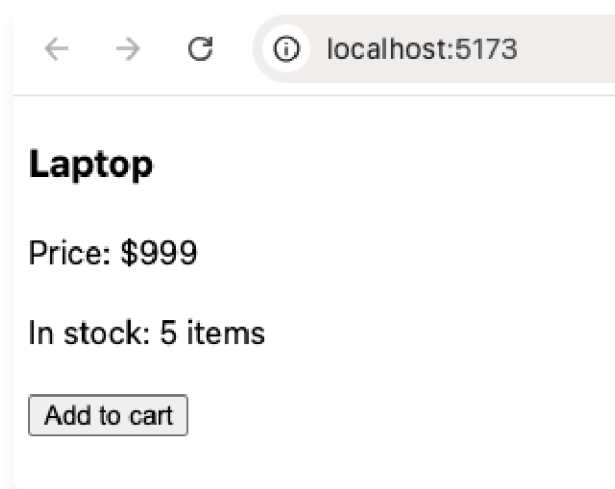
```
import ProductCard from "../components/ProductCard";

const product = {
  id: 1,
  name: "Laptop",
  price: 999,
  stock: 5,
};

const App = () => {
  return <ProductCard product={product} />;
};

export default App;
```

Now, If you check the application, you will see the following output:



As you can see, we correctly see the translation and interpolated values from the `translation.js` file.

Multiple Variables in One String

You can use multiple variables in a single translation string:

```
// Translation string
"greeting": "Hello {{firstName}} {{lastName}}, you have {{count}} new messages."

// Usage in code
t('greeting', {
  firstName: 'John',
  lastName: 'Doe',
  count: 5
})
// Output: "Hello John Doe, you have 5 new messages."
```

Interpolation with Counts (Pluralization)

One of the most powerful features is combining interpolation with pluralization. i18next automatically handles plural forms based on the count:

```
{
  "items_zero": "You have no items",
  "items_one": "You have {{count}} item",
  "items_other": "You have {{count}} items"
}
```

When you use the `count` parameter, i18next automatically selects the correct plural form:

```
t('items', { count: 0 }) // "You have no items"
t('items', { count: 1 }) // "You have 1 item"
t('items', { count: 5 }) // "You have 5 items"
```

Different languages have different pluralization rules. For example:

- **English:** zero, one, other
- **Polish:** zero, one, few, many, other
- **Arabic:** zero, one, two, few, many, other

i18next handles these differences automatically!

Custom Formatting with Interpolation

You can also apply formatting to interpolated values. This is particularly useful for dates, currencies, and numbers:

```
"price": "Price: {{amount, currency}}"
"lastLogin": "Last login: {{date, date}}"
```

Usage:

```
t('price', { amount: 99.99 })
// Output with formatting: "Price: $99.99" (in English)
// Output with formatting: "Price: 99,99 €" (in French)

t('lastLogin', { date: new Date() })
// Output: "Last login: December 4, 2024" (in English)
// Output: "Last login: 4 décembre 2024" (in French)
```

If you remember, we added a `format` function inside the `utils/i18n.js` file because of which we see the \$ in `$99.99` and `€` in `99,99 €` if the user's language is french `fr`.

If you want to see that in action, open `src/main.jsx` file and add the following import at the top of the file:

```
import i18next from "i18next";
```

and call the `changeLanguage` method from `i18next` by passing `fr` as the value.

Here's the complete code:

```

import i18next from "i18next"; // import i18next package
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App.jsx";
import "../index.css";
import "../utils/i18n";

i18next.changeLanguage("fr"); // change language to french

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);

```

And as a temporary code you can add the following JSX inside the `ProductCard.jsx` file:

```

return (
  <div>
    {/* your old JSX */}

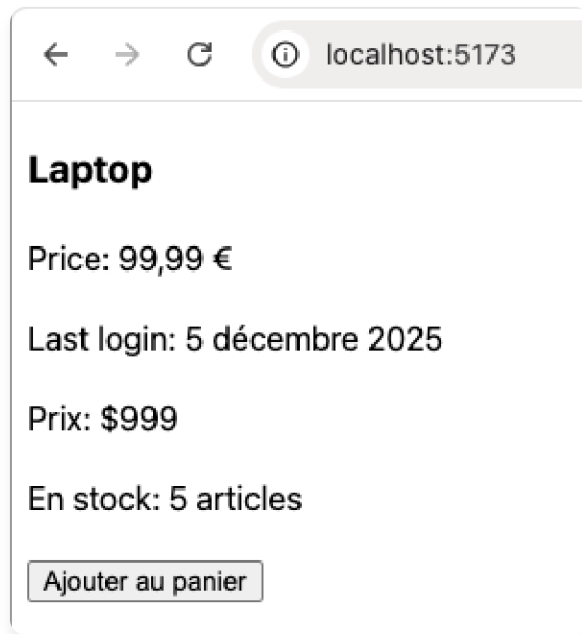
    <p>{t("price", { amount: 99.99 })}</p>

    <p>{t("lastLogin", { date: new Date() })}</p>

    {/* your new JSX */}
  </div>
);

```

Now, If you check the application, you will the following output:



Important Notes About Interpolation

- **Escaping:** By default, i18next doesn't escape HTML in interpolated values because React already handles escaping. This is why we set `escapeValue: false` in the `utils/i18n.js` file.
- **Missing Variables:** If you forget to pass a required variable, i18next will leave the placeholder as it is in the output so never forget to pass the value for placeholder

```
t('welcome', {})
```

 // Output: "Welcome, {{name}}!"

- **Default Values:** You can provide fallback values:

```
t('welcome', { name: userName || 'Guest' })
```

- **Performance:** Interpolation is very fast, so don't worry about using it extensively in your app.

Now that you understand how variables and interpolation work, you're ready to use them effectively in your components!

Now, create a `UserProfile.jsx` file inside the `components` folder and add the following contents inside it.


```
import { useTranslation } from "react-i18next";

function UserProfile({ user }) {
  const { t } = useTranslation();

  return (
    <div>
      <h2>{t("profile.welcome", { name: user.name })}</h2>

      <p>{t("profile.loyaltyPoints", { points: user.points })}</p>
    </div>
  );
}

export default UserProfile;
```

Now, let's use this component inside the `App.jsx` file:

```
import ProductCard from "../components/ProductCard";
import UserProfile from "../components/UserProfile";

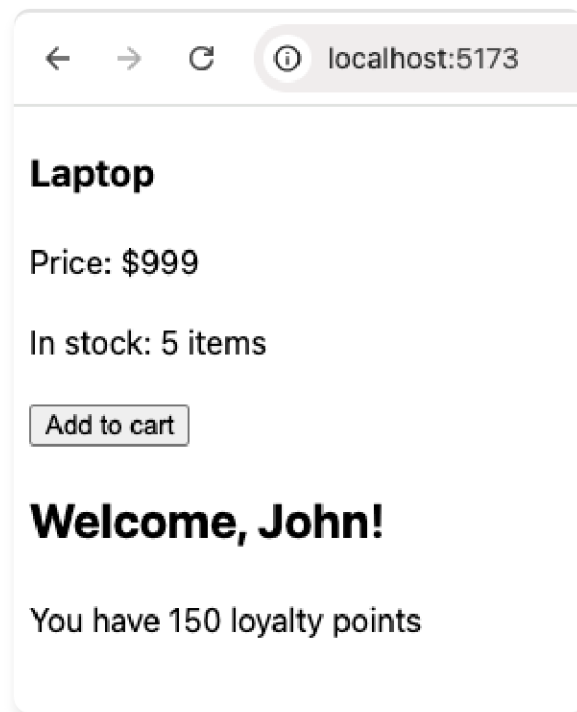
const product = {
  id: 1,
  name: "Laptop",
  price: 999,
  stock: 5,
};

const user = {
  name: "John",
  points: 150,
};

const App = () => {
  return (
    <
      <ProductCard product={product} />
      <UserProfile user={user} />
    </>
  );
};

export default App;
```

If you check the application now, you will see the following output:



How to Build a Language Switcher

A language switcher allows users to change the application language. Here's how to build one.

Inside `components` folder create a new file with name `LanguageSwitcher.jsx` and add the following content inside it:

```

import { useTranslation } from "react-i18next";

const languages = [
  { code: "en", name: "English", flag: "🇺🇸" },
  { code: "es", name: "Español", flag: "🇪🇸" },
  { code: "fr", name: "Français", flag: "🇫🇷" },
];

function LanguageSwitcher() {
  const { i18n } = useTranslation();

  const changeLanguage = (languageCode) => {
    i18n.changeLanguage(languageCode);
  };

  return (
    <div
      style={{
        display: "flex",
        gap: "10px",
        padding: "10px",
        background: "#f5f5f5",
        borderRadius: "8px",
      }}
    >
      {languages.map((lang) => (
        <button
          key={lang.code}
          onClick={() => changeLanguage(lang.code)}
          style={{
            padding: "8px 16px",
            background: i18n.language === lang.code ? "#2196f3" : "#fff",
            color: i18n.language === lang.code ? "#fff" : "#000",
            border: "1px solid #ddd",
            borderRadius: "4px",
            cursor: "pointer",
            display: "flex",
            alignItems: "center",
            gap: "8px",
            fontSize: "14px",
            fontWeight: i18n.language === lang.code ? "bold" : "normal",
            transition: "all 0.2s ease",
          }}
        >
          <span>{lang.flag}</span>
          <span>{lang.name}</span>
        </button>
      ))}
    </div>
  );

```

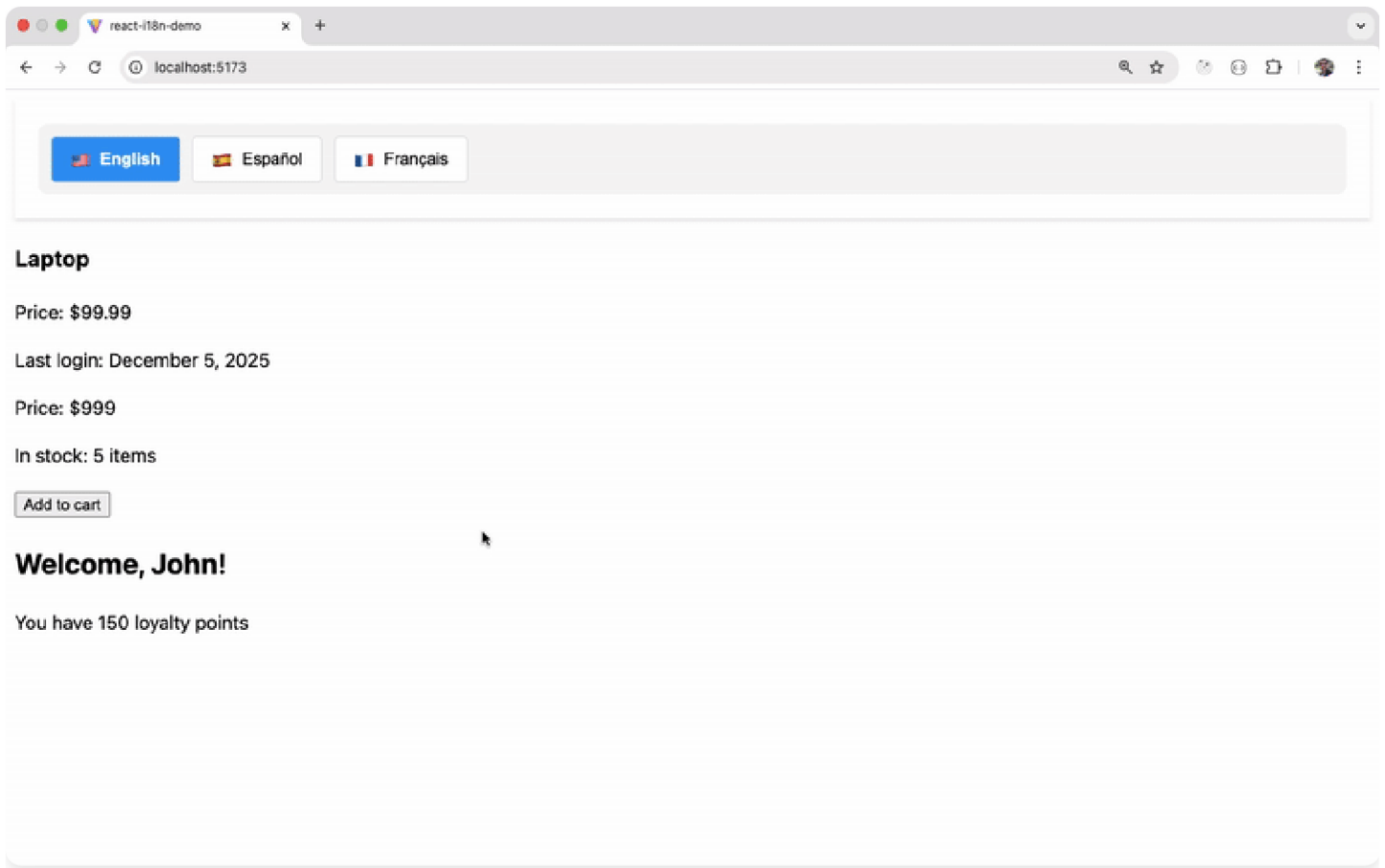
```
}  
  
export default LanguageSwitcher;
```

Complete App with Language Switcher

Here's a complete example showing the language switcher in action:

```
import LanguageSwitcher from "../components/LanguageSwitcher";  
import ProductCard from "../components/ProductCard";  
import UserProfile from "../components/UserProfile";  
  
const product = {  
  id: 1,  
  name: "Laptop",  
  price: 999,  
  stock: 5,  
};  
  
const user = {  
  name: "John",  
  points: 150,  
};  
  
const App = () => {  
  return (  
    <  
      <header  
        style={{  
          padding: "20px",  
          background: "#fff",  
          boxShadow: "0 2px 4px rgba(0,0,0,0.1)",  
          marginBottom: "20px",  
        }}  
      >  
        <LanguageSwitcher />  
      </header>  
      <ProductCard product={product} />  
      <UserProfile user={user} />  
    </>  
  );  
};  
  
export default App;
```

If you check the application you can see the working language switcher as shown below:



Conclusion

Congratulations! You've learned how to build a fully internationalized React application.

You can find the complete source code for this application here: <https://github.com/myogeshchavan97/react-i18n-demo>

In this tutorial, you learned:

- What internationalization (i18n) is and why it matters
- How to set up react-i18next in your React app
- How to organize translations in different files
- How variables and interpolation work in translations
- How to use the `useTranslation` hook for translations
- How to handle pluralization automatically
- How to format dates, numbers, and currencies for different locales
- How to build a language switcher component

-
- Follow me on LinkedIn: <https://www.linkedin.com/in/yogesh-chavan97/>
 - Ultimate React Ebooks Collection: <https://courses.yogeshchavan.dev/the-ultimate-react-ebooks-components-hooks-redux-routing-and-more>
 - All my courses,ebooks, webinars: <https://courses.yogeshchavan.dev/>

This PDF is generated using PDFMaker (<https://www.pdfmaker.cc/>)